

Smart AirGuard: A Preliminary Edge-Cloud IIoT Architecture for Gas Monitoring in Enclosed Automotive Workspaces

Darya Martsinetskaya*, João-Paulo Barros*[†] and João Carlos Martins*

**School of Technology and Management*

Polytechnic Institute of Beja, Beja, Portugal

[†]Center of Technology and Systems (UNINOVA-CTS) and LASI, Portugal

Emails: 26610@stu.ipbeja.pt, joao.barros@ipbeja.pt, joao.martins@ipbeja.pt

Abstract—This Work-in-Progress paper presents Smart AirGuard, a low-cost Industrial Internet of Things (IIoT) prototype for monitoring hazardous exhaust-related gases and activating local ventilation in enclosed automotive workspaces. The system follows a hybrid edge-cloud architecture: safety-critical actions, such as fan activation and local alarms, run directly on an ESP32-based edge node, while cloud services support remote monitoring, notifications, data logging, and preliminary trend analysis. The prototype combines broad-spectrum gas sensing, temperature and humidity sensing, occupancy detection, MQTT communication, Node-RED orchestration, and Telegram notifications. Preliminary validation was conducted in a 4500 m³ commercial automotive workshop and through controlled stress tests involving exhaust exposure, aerosols, oil vapours, high humidity, sub-zero temperature, and thermal stress. Under the tested conditions, the prototype detected all gas events above the configured threshold, activated ventilation in less than one second, and delivered remote alerts within one to two seconds, with no unwarranted activations during selected non-critical exposures. The results support the feasibility of an edge-based safety architecture, while highlighting the need for calibrated gas-specific sensing, longer-term trials, and validated predictive analytics.

Index Terms—IoT, gas monitoring, air quality, ESP32, MQTT, Node-RED, Telegram Bot, occupational safety

I. INTRODUCTION

Hazardous exhaust gases can accumulate quickly in enclosed automotive spaces such as garages and workshops, especially when ventilation is poor. Carbon monoxide (CO) is particularly dangerous because it is colourless, odourless, and tasteless, allowing harmful exposure without immediate warning signs [1]. Since vehicle engines running in closed garages can raise CO concentrations far above recommended limits [2], fast detection and local response are essential.

Low-cost microcontrollers, wireless communication, and cloud dashboards have made IoT-based air-quality monitoring more accessible. However, many systems remain reactive, activating alarms or ventilation only after a threshold is exceeded, and are often tested in controlled indoor settings rather than in demanding automotive workshops, where exhaust gases,

aerosols, oil vapours, humidity, temperature changes, and intermittent occupancy may affect sensor behaviour.

This Work-in-Progress paper presents Smart AirGuard, a low-cost edge-cloud IoT prototype for gas monitoring and automated ventilation in enclosed automotive workspaces. The system combines an ESP32 edge node, gas and environmental sensing, occupancy detection, local alarms, fan actuation, MQTT communication, Node-RED dashboards, and Telegram notifications. Critical safety actions run locally and do not depend on cloud connectivity, while cloud components support monitoring, logging, user interaction, and preliminary trend analysis. The 15–30 min forecast component is not validated in this WiP paper. The architecture is prepared for future predictive analytics, but the validation reported here focuses on gas-event detection, local response, and preliminary behaviour in realistic conditions.

The study is guided by five research questions: RQ1: How does a low-cost gas sensor behave when exposed to common interferences found in a real automotive workshop?

RQ2: Can multi-parameter sensing, including gas, temperature, humidity, and occupancy, improve alert interpretation and reduce unwarranted activations?

RQ3: What latency can be achieved between hazard detection, local actuation, and remote user notification?

RQ4: What safety benefits result from executing critical control logic locally at the edge rather than relying exclusively on cloud services?

RQ5: What trade-offs emerge between hardware cost, system complexity, and deployability for small-scale automotive environments?

The main contributions are: (1) a hybrid edge-cloud architecture that separates local safety actions from cloud-based monitoring and analysis; (2) a context-aware alerting strategy combining gas concentration, environmental variables, and occupancy information; (3) a preliminary evaluation in a commercial automotive workshop, complemented by controlled stress tests; and (4) an empirical discussion of limitations observed with low-cost sensing hardware, especially gas selectivity and thermal robustness.

The preliminary results suggest that an affordable edge-

This work was supported by FCT – Fundação para a Ciência e a Tecnologia within the scope of the CTS Research Unit – Center for Technology and Systems/UNINOVA/FCT/NOVA, under reference UID/00066/2025.

based IoT system can support timely gas-event detection and automatic ventilation in enclosed automotive workspaces. They also indicate the need for further work on calibrated gas-specific sensing, longer-term field trials, and experimentally validated predictive models.

II. RELATED WORKS

Several IoT-based gas monitoring systems have been proposed for domestic and industrial safety. Balogun et al. [3] integrated an MQ-135 sensor with a NodeMCU for LPG cylinder monitoring, including predictive consumption analysis. Muhammad et al. [4] demonstrated automatic solenoid valve control and email alerts using Arduino and Cayenne IoT. Niveditha et al. [5] combined MQ-2/MQ-6 sensors with voice alerts and exhaust fan control. Flores-Cortez et al. [6] implemented an AWS-based system for LPG and CO monitoring using MQ-5 and MQ-9 sensors. Batra et al. [7] developed SmartCOdetect, an Arduino-based system that automatically opens car windows upon CO detection.

However, most existing systems are designed for LPG leaks in kitchens rather than automotive exhaust gases in enclosed workshops. Many rely on cloud-dependent decision-making, introducing latency and safety risks. Predictive analytics are rarely validated in real workshop conditions. Furthermore, few systems address context-aware alerting to reduce false positives in harsh environments.

III. SYSTEM ARCHITECTURE

Smart AirGuard implements a distributed IoT architecture, as illustrated in Fig. 1, integrating embedded hardware, local processing, cloud services, and user interaction layers.

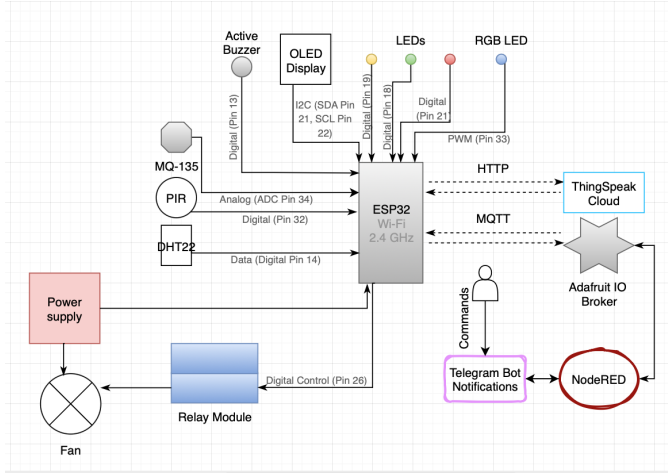


Fig. 1. Smart AirGuard architecture.

A. Hardware Subsystem

The ESP32 serves as the central processing unit. The sensing array comprises an MQ-135 gas sensor, DHT22 temperature and humidity sensor, and HC-SR501 passive infrared (PIR) motion sensor. The actuation module features a direct current (DC) cooling fan operated via a 2-channel relay, an

active buzzer for audible alerts, and visual indicators including discrete Light-Emitting Diodes (LEDs) for gas danger (red), temperature out-of-range (yellow), and high humidity (green), plus a Red Green Blue (RGB) LED for motion status. In this work, MQ-135 readings are used as relative gas concentration estimates derived from the sensor calibration curve and are employed solely for threshold-based event detection rather than quantitative gas-species measurement.

B. Communication and Software Framework

The system utilizes a hybrid communication topology. The ESP32 publishes sensor data via the Message Queuing Telemetry Transport (MQTT) protocol through Adafruit IO using an event-driven transmission mechanism, whereby data are transmitted when the temperature changes by more than 0.5°C, the humidity changes by more than 1.0%, the gas concentration changes by more than 50 ppm, or when a system state transition occurs. A mandatory heartbeat payload is transmitted every 60 seconds. Concurrently, Hypertext Transfer Protocol (HTTP) GET requests are sent to ThingSpeak every 15 seconds to update eight telemetric fields corresponding to temperature, humidity, gas concentration in parts per million, motion detection, gas alerts, temperature alerts, humidity alerts, and fan status. Through ThingSpeak's MATLAB integration, weekly analytical processing is conducted to calculate mean, maximum, and minimum gas concentration values, perform threshold-based anomaly detection, and generate visual reports.

Node-RED, deployed locally via Docker, acts as the central orchestration engine. Key computational nodes include:

- *Data ingestion and dashboard visualization* are handled via Node-RED with MQTT subscription and real-time gauges.
- *Predictive Analytics (ongoing work, not validated)*: Maintains a buffer of the last 12 readings per sensor for future 15- and 30-minute forecasts.
- *Telegram Integration*: Polls Telegram getUpdates every 2 seconds, maps commands to MQTT feeds, formats responses, and automatically forwards critical alerts without user request.
- *Simulation Framework*: Ten simulated scenarios with automated test sequencing and a test mode controller that bypasses physical MQTT input.
- *Model Validation*: Computes Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and Accuracy in real time by comparing predictions against actual readings.

C. Alerting Protocol

A priority-based alerting matrix governs system responses with three configurable thresholds: gas at 1000 ppm, temperature at 10–35°C, and humidity at 90%.

Gas-related events maintain the highest priority. Exceeding the 1000 ppm threshold forces immediate fan activation, audible/visual alarms, and instant Telegram notifications, strictly overriding all manual inputs.

Temperature and humidity threshold crossings activate yellow and green LEDs and Telegram notifications but no actuation unless accompanied by gas exceedance.

Manual fan commands (dashboard or Telegram) temporarily override automatic control, auto-reverting after 5 minutes of inactivity. During active gas emergencies, manual OFF commands are blocked to ensure ventilation cannot be disabled while hazardous conditions persist.

IV. PRELIMINARY EXPERIMENTAL VALIDATION

To evaluate the system under representative operating conditions, a series of controlled experiments was conducted in both commercial garage environments and isolated stress-testing setups. These tests were designed to validate threshold sensitivity, actuator latency, hazard prioritization, and algorithmic resilience.

A. Test Environment

Experimental validation was conducted in a commercial garage workshop (4500 m³) located in Estombar, Portugal, used for moto and car maintenance. The garage has no built-in mechanical ventilation, natural ventilation is controlled via a manual roller door (3×4 m) which can be fully open, half open, or fully closed. Occupancy during testing consisted of 2-3 persons performing routine maintenance work. Baseline environmental conditions at the start of testing were: temperature 23.6°C, humidity 57.7%, gas concentration approximately 400 ppm (normal baseline). The Smart AirGuard prototype was placed on a table inside the workspace and powered by an autonomous power bank, with all sensors and actuators operational. The prototype operated continuously over a four-hour monitoring period, powered autonomously.

B. Garage Trial Results

Seven controlled tests were conducted using a 125 cc scooter as the exhaust source, a gas lighter, an aerosol spray, and motor oil vapours. The scooter was positioned at distances ranging from maximum proximity (<0.1 m) to 1.5 m from the sensor. The garage door was configured in three positions: fully open, half open, and fully closed.

System performance metrics included:

- *Absence of False Positives:* Unwarranted actuation did not occur during non-critical exposures (911 ppm scooter at 1.5m, 405 ppm aerosol, 410 ppm oil vapours).
- *Reliable Hazard Detection:* Gas concentrations exceeding 1000 ppm invariably triggered the correct safety protocols. Minimal exceedance (only +8 ppm above threshold) activated the fan and buzzer within less than 1 second. Higher concentrations - 1100 ppm (half-open door), 1350 ppm (fully closed door), 1687 ppm (lighter gas) - all produced reliable activation with sub-second latency.
- *Ventilation-Dependent Recovery:* Recovery time to baseline (400 ppm) varied directly with natural ventilation availability. With the door fully open, recovery took 25 seconds (1008 ppm) or only 7 seconds (1687 ppm, due to steeper concentration gradient). With the door half

open, recovery extended to 45 seconds. With the door fully closed - eliminating natural ventilation entirely - the system relied solely on mechanical ventilation, requiring 5 minutes to clear the 1350 ppm peak. No observable baseline shift was detected during the conducted experiments.

C. Controlled Environmental Stress Testing

To ascertain system robustness, boundary-condition stress tests were conducted:

- *Multi-Hazard Conflict (100% Humidity + Gas):* The system correctly prioritised gas alerts over humidity.
- *Sub-Zero Operation (-5.5°C):* The prototype was placed inside a freezer compartment, reaching -5.5°C. Three separate gas trigger events were induced using a lighter. The MQ-135 sensor successfully detected gas peaks of 1197 ppm, 1233 ppm, and 1999 ppm - the highest recorded concentration across all experiments. The fan activated automatically for each trigger, and temperature alerts triggered when readings fell below 10°C. Condensation during warm-up caused humidity to spike to 99.8%, activating humidity alerts. All sensors recovered to normal operation after the test without permanent drift, confirming reliable operation at sub-freezing temperatures.
- *Thermal Stress and Hardware Limitations (54.2°C):* A household hair dryer was directed at the prototype, raising temperature to 54.2°C. The system correctly detected the 35°C threshold and activated temperature alerts. A gas trigger at 1076 ppm successfully activated the fan. A critical finding is that the DHT22 sensor stopped providing valid readings at approximately 42°C and required a manual power cycle to recover - even after temperature returned to 22.6°C. By contrast, the MQ-135 gas sensor continued functioning normally throughout. This identifies a clear hardware limitation for deployments in high-temperature environments (e.g., summer workshops, tropical climates), with recommended alternatives being DS18B20 or BME280 sensors plus a watchdog timer for automatic reset.

D. Evaluation of the Local Trend Estimation Module

Note: This subsection validates short-term (30 s) predictions, not the 15–30 min forecasts (Section III-B).

The Model estimation framework continuously monitored prediction accuracy by comparing actual gas readings against predicted values (based on readings from six samples prior, approximately 30 seconds). Four metrics were tracked: MAE (target <50 ppm), RMSE (target <75 ppm), MAPE (target <15%), and overall Accuracy (target >80%).

Under normal conditions, the framework maintained MAE below 15 ppm, RMSE below 25 ppm, MAPE below 5%, and Accuracy above 95% - substantially exceeding all targets. During sudden gas spikes, temporary prediction errors occurred: MAE increased to 40-80 ppm, RMSE to 65-120 ppm, and Accuracy dropped to 65-85%. These transient errors

are expected, as the model cannot anticipate abrupt pollutant releases. Following gas normalisation, all metrics returned to baseline levels within 30-60 seconds (garage tests) or 60-90 seconds (freezer test at -5.5°C). The framework successfully distinguished between genuine combustion events (triggering temporary error spikes) and non-combustion sources (where metrics remained stable). Critically, the model demonstrated robust recovery with no systemic degradation.

E. Telegram and Multi-Channel Notifications

Telegram Bot integration delivered all alerts (gas, temperature, humidity, motion, fan state) within 1–2 seconds. Users could query data and control the fan via commands (/status, /fan on/off/auto) with sub-second latency. The system successfully notified for all trigger events across freezer (-5.5°C), heat (54.2°C), and combined humidity-gas tests, with zero failures over 4 hours of testing. This indirect architecture provides algorithmic resilience without burdening the microcontroller.

V. SYSTEM LIMITATIONS AND FUTURE WORK

This preliminary research phase has identified specific areas requiring refinement prior to industrial scale-up:

- *Thermal Sensor Vulnerability* – The DHT22 sensor stopped providing valid readings at approximately 42°C and required manual power cycle recovery. Possible improvement is replacement with DS18B20 or BME280 temperature sensors and implementation of watchdog timer for automatic ESP32 reset upon sensor failure detection.
- *Sensor Selectivity* – The sensor responds to a broad range of reducing gases and provides CO_2 -equivalent readings rather than specific gas identification. Possible improvement is adding selective sensors or implementation of sensor fusion with machine learning for gas classification.
- *Humidity recovery with active moisture source* – When a humidifier remained active, fan operation could not rapidly reduce humidity until source removal. Possible improvement is integration of humidity source detection logic and user recommendations.
- *Edge Computing Independence* – While critical safety responses operate without internet, Machine Learning (ML) predictions and Telegram notifications require connectivity. Possible improvement is to implement TinyML for on-device inference. Unlike the cloud-dependent 15–30 min forecasts, the short-term (30 s) prediction already runs locally on the ESP32.

VI. DEPLOYMENT ARCHITECTURES AND ECONOMIC FEASIBILITY

Three deployment options have been evaluated for commercial viability, assuming a baseline hardware cost of €31.01 per unit:

- Option 1 (ThingSpeak only): €31.01 hardware + €210.24/year. Basic monitoring, no MQTT or ML.

- Option 2 (Node-RED + Adafruit IO+ + Telegram): €31.01 + €85.15/year. Full functionality (MQTT, ML, custom dashboards).
- Option 3 (Node-RED on Virtual Private Server (VPS) + Telegram): €31.01 + €96/year for VPS. Full functionality, no third-party subscriptions.

The Telegram Bot API remains free for commercial use.

VII. CONCLUSION

This Work-in-Progress paper presented Smart AirGuard, an IoT-based gas monitoring and automated ventilation system for enclosed automotive environments. Based on preliminary validation, the core research questions are answered as follows:

- 1) Across the seven evaluated scenarios, all gas events above the configured threshold were detected and no unwarranted activations were observed.
- 2) Multi-parameter sensing improved detection accuracy by enabling priority-based alerting-gas emergencies overrode humidity alerts, and PIR occupancy provided context for notification urgency.
- 3) Actuator activation occurred in <1 second, and Telegram notifications were delivered within 1-2 seconds, satisfying real-time safety requirements.
- 4) Critical responses (fan, buzzer) executed locally on ESP32 remained operational even in the absence of cloud connectivity, while cloud-based Node-RED provides dashboards (the 15–30 min forecasts remain unvalidated ongoing work).
- 5) Three commercial architectures are identified: ThingSpeak-only, Node-RED+Adafruit IO+Telegram, self-hosted Node-RED on VPS.

Current limitations are identified with concrete improvements discussed. Smart AirGuard provides an affordable, reliable, and extensible platform for reducing exposure risks associated with exhaust-related gases in industrial workspaces and commercial garages.

REFERENCES

- [1] GBD 2021 Carbon Monoxide Poisoning Collaborators, “Global, regional, and national mortality due to unintentional carbon monoxide poisoning, 2000–2021: results from the global burden of disease study 2021,” *The Lancet Public Health*, vol. 8, no. 11, pp. e839–e849, Oct 2023.
- [2] T. Teleszewski and K. Gladyszewska-Fiedoruk, “Carbon monoxide concentration in the garage of a single-family house-experiment and one-dimensional model of carbon monoxide concentration,” *Applied Sciences*, vol. 15, no. 3, p. 1146, Jan 2025.
- [3] D. Balogun, S. Shamim, U. Sipai, N. Kothari, T. Trivedi, and V. Patel, “Lpg smart guard: An iot-based solution for real-time gas cylinder monitoring and safety in smart homes,” *Engineering Proceedings*, vol. 82, no. 9, Nov 2024.
- [4] F. R. Muhammad, A. S. Putra, and A. Aribowo, “Arduino-based gas leakage monitoring and treatment system,” *2022 1st International Conference on Technology Innovation and Its Applications (ICTIIA)*, Nov 2022.
- [5] N. Niveditha, B. Harshini, and M. P. Mahalakshmi, “Smart gas leakage detection using iot,” *International Research Journal on Advanced Engineering Hub (IRJAEH)*, vol. 3, pp. 687–692, Mar 2025.
- [6] O. O. Flores-Cortez, R. Cortez, and B. Gonzalez, “Design and implementation of an iot based lpg and co gases monitoring system,” Jun 2021.
- [7] N. Batra, J. Kaur, and N. K. Batra, “Smartcodetect: An automated car window opening system on detection of carbon monoxide,” *International Journal of Vehicle Structures and Systems*, vol. 12, no. 4, pp. 398–404, 2020.